

GESTÃO E QUALIDADE DE SOFTWARE



TÉCNICAS DE TESTES ÁGEIS



TÉCNICAS DE TESTES ÁGEIS



O que é?

- Práticas utilizadas em ambientes de desenvolvimento ágil para garantir a qualidade contínua do software ao longo dos ciclos de entrega curtos.



Algumas técnicas

- Teste Baseado em História de Usuário
- BDD (Behavior-Driven Development)
- TDD (Test-Driven Development)

Teste Baseado em História de Usuário



Objetivo: Validar se as funcionalidades implementadas atendem às histórias de usuário.

Como funciona: Os critérios de aceitação são usados como base para criar os testes.



Exemplo

Dado um usuário logado,
quando ele clica em “Sair”,
então ele deve ser redirecionado para a tela de login.

História de Usuário



Descrições curtas e simples de um recurso contadas da perspectiva da pessoa(usuário ou cliente) que deseja o novo recurso.

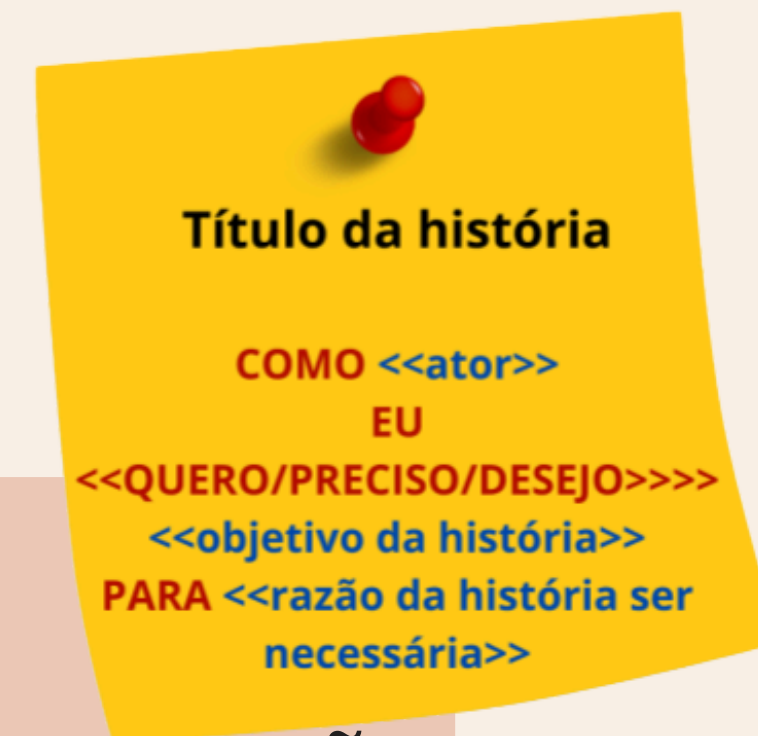
O estilo de redação é **livre**, não há um modelo padronizado.



Escritas no espaço de um cartão (notas adesivas)

Um modelo simples:

Como(papel), **eu quero**(algo), **para** (benefício).



História de Usuário



FAQ's

COMO visitante do site, EU QUERO ler as perguntas frequentes PARA obter respostas rápidas.

FORMAS DE PAGAMENTO

COMO cliente, EU QUERO que sejam disponibilizadas diversas formas de pagamento PARA pagar meu pedido.

CONSULTA DE PREÇOS

COMO vendedor, EU PRECISO consultar o preço de um produto PARA que eu possa informar ao cliente.

Teste de Aceitação



Descreve os comportamentos que o sistema em desenvolvimento necessita para atender às regras de negócio do cliente.



Dado (uma pré-condição),
quando (o usuário realiza uma ação) **então** (esse é o resultado esperado).

Vendedor Pesquisar por
Código do Produto

DADO QUE eu esteja logado
como vendedor, QUANDO eu
pesquisar pelo código "0054"
ENTÃO eu devo ver o produto
relacionado à busca.

TDD (Test-Driven Development)



Técnica onde você escreve primeiro os testes, depois o código que faz esses testes passarem.



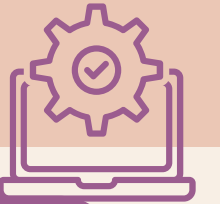
Ciclo básico:

Red – Escreva um teste e veja ele falhar.

Green – Escreva o código mínimo para passar no teste.

Refactor – Melhore o código, mantendo os testes passando.

PADRÃO AAA (ARRANGE - ACT - ASSERT)



Arrange
(Preparar)

Configure os dados e objetos necessários.

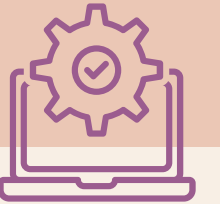
Act
(Agir)

Execute o método ou ação a ser testada.

Assert
(Verificar)

Verifique se o resultado é o esperado.

Notações importantes em Java



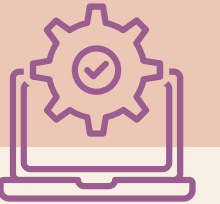
@Test

Marca o método com um teste.

**assertTrue(),
assertFalse(),
assertEquals()**

Métodos de verificação (assertivas).

Passo 1: Escrever o teste (RED)



Criar um método que verifica se um número é par.

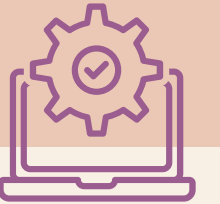
```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

@Test
void deveRetornarTrueParaNumeroPar() {
    // Arrange
    int numero = 4;

    // Act
    boolean resultado = Validador.ePar(numero);

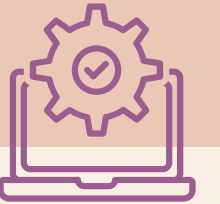
    // Assert
    assertTrue(resultado);
}
```

Passo 2: Escrever o código (GREEN)



```
public class Validador {  
    public static boolean ePar(int numero) {  
        if (numero % 2 == 0) {  
            return true;  
        } else {  
            return false;  
        }  
    }  
}
```

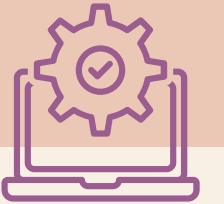
Passo 3: Refatorar (REFACTOR)



- Deixe o código mais simples, sem alterar o comportamento.
- Mantenha os testes passando como garantia de que nada foi quebrado.
- Pode incluir melhoria de nomes, extração de métodos, remoção de duplicidade, etc.

```
public class Validador {  
    public static boolean ePar(int numero) {  
        return numero % 2 == 0;  
    }  
}
```

BDD (Behavior-Driven Development)



Prática de desenvolvimento ágil baseada na descrição de comportamentos esperados do sistema em linguagem natural estruturada (Gherkin).



Ciclo básico:

Discovery: Conversa entre time técnico e negócio sobre o que deve ser feito

Formulation: Escrita de cenários de comportamento com Gherkin.

Automation: Implementação dos testes automatizados (ex: com Cucumber)

GHERKIN



O que é?

- um formato que podemos utilizar para padronizar, documentar, e até reciclar código e funcionalidades.
- padronizar a forma de descrever especificações de cenários, baseado na regra de negócio.



Criada por Aslak Hellesoy

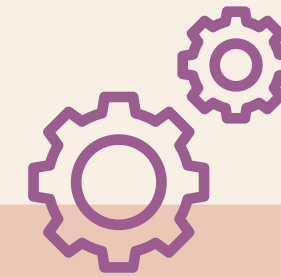
- 2003: como o TDD (Test Driven Development) poderia ser melhor aplicado.
- 2008: Nasce Cucumber e a linguagem Gherkin.

GHERKIN



Linguagem natural

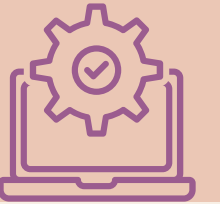
- forma de entendimento e compreensão acerca das especificações levantadas a partir da perspectiva do *stakeholder*.



Característica

- não necessita de detalhes de implementação.
- orientada a espaços, ela usa indentação para definir a estrutura.
- traduzido para **mais de 70 idiomas**.

GHERKIN



Como?

- Ele é escrito em forma de “steps” (ou “passos”), os quais especificam cada etapa de interação do usuário com o sistema a ser testado.



Importância

- é um dos elementos principais quando se trata de BDD (Behavior-Driven Development) em automação.

GHERKIN



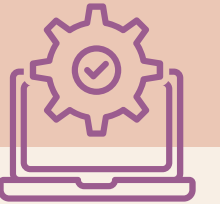
Gherkin usa um conjunto de palavras - chave especiais para dar estrutura e significado às especificações executáveis.

- Cada palavra-chave é traduzida para vários idiomas falados.



Os comentários são permitidos apenas no início de uma nova linha, em qualquer lugar no arquivo (#).

GHERKIN - Palavras-chave de Steps



01

Given
(Dado)

02

When
(Quando)

03

Then
(Então)

04

And
(E)

05

But
(Mas)

05

*

Palavras-chave



Given (Dado)

- Descrever o contexto inicial do sistema - a cena do cenário.
- Normalmente é algo que aconteceu no passado.
- Colocar o sistema em um estado conhecido.
- Utilizado para especificar uma pré condição.

When (Quando)

- Descrever um evento ou uma ação.
- Pode ser uma pessoa interagindo com o sistema ou pode ser um evento acionado por outro sistema.
- É altamente recomendável que você tenha apenas uma única When etapa por cenário.

Palavras-chave



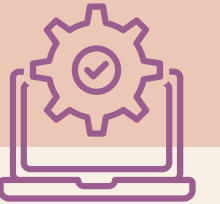
Then (Então)

- descrever um resultado ou resultado esperado.
- afirmação para comparar o resultado real (o que o sistema realmente faz) com o resultado esperado (o que a etapa diz que o sistema deve fazer).
- Um resultado deve ser uma saída observável.

And (E)

- Utilizado para caso seja necessário mais uma interação com o sistema para complementar um fluxo, mas que não necessariamente se trata de uma ação ou reação.

Palavras-chave



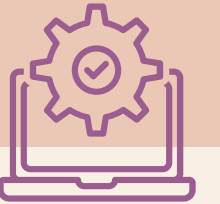
But
(Mas)

- serve a mesma funcionalidade do **And**, porém é normalmente utilizado após uma validação negativa depois do **Then**.

*

- serve para ser usado no lugar de qualquer uma das palavras-chave da etapa normal. Isso pode ser útil quando você tem algumas etapas que são efetivamente uma lista de coisas

Exemplo: Realizar Login



1. **Pré condição:** Possuir uma conta no sistema.
2. Acessar a página de login
3. Preencher credenciais
4. Clicar no botão de login
5. Esperar o login ser completado

1. **Dado** que “Fulano” possui uma conta no sistema
2. **E** ele acessa a página de login
3. **E** ele preenche suas credenciais válidas
4. **Quando** ele aciona a opção de realizar login
5. **Então** ele deve ser redirecionado para a página inicial logado

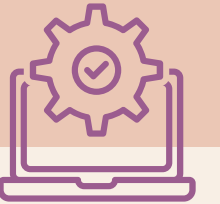


Até agora é puro texto!
Mas como
escrever/programar/desenvolver
isso tudo?

É verdade, por esse motivo
vamos agora entender as
"Features" (ou
Funcionalidades).



GHERKIN - Palavras-chave de funcionalidade



01 Feature
(Funcionalidade)

02 Rule
(Regra de negócio)



Gherkin 6

03 Example/Scenario
(Exemplo/Cenário)

04 Scenario Outline/
Template
(Esquema de Cenário)

10 Backgroud
(Contexto)

Palavras-chave de funcionalidade



Feature (Funcionalidade)

- Fornecer uma descrição de alto nível de um recurso de software e agrupar cenários relacionados.
- A primeira palavra-chave primária em um documento Gherkin deve ser sempre **feature**, seguida por um **:** e um texto curto que descreve o recurso.

Palavras-chave de funcionalidade



Rule
(Regra de negócio)

- Representar uma **regra de negócios** que deve ser implementada.
- Fornece informações adicionais para uma funcionalidade.
- É usada para agrupar vários cenários que pertencem a esta regra de negócios .
- Deve conter um ou mais cenários que ilustram a regra específica.

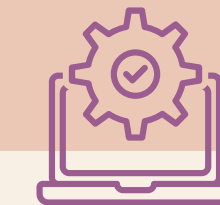
Palavras-chave de funcionalidade



Example/Scenario (Exemplo/Cenário)

- Especificação executável do sistema
- Você pode ter quantas etapas quiser, mas o recomendado são de 3 a 5 etapas.
- Segue este mesmo padrão:
 - Descreva um contexto inicial (**Given** etapas)
 - Descreva um evento (**When** etapas)
 - Descreva um resultado esperado (**Then** etapas)

Palavras-chave de funcionalidade



Scenario Outline/ Template (Esquema de Cenário)

- executar as mesmas **Scenario** várias vezes, com diferentes combinações de valores.
- deve conter uma seção **Examples**(ou **Scenarios**).
- Suas etapas são interpretadas como um modelo que nunca é executado diretamente. Em vez disso, o **Scenario Outline** é executado uma vez para cada linha na seção **Examples** abaixo dele (sem contar a primeira linha do cabeçalho).
- As etapas podem usar parâmetros **<>** delimitados que fazem referência aos cabeçalhos na tabela de exemplos.

Palavras-chave de funcionalidade



Background (Contexto)

- colocado antes do primeiro Scenario, no mesmo nível de recuo.
- Usado para quando se tem as mesmas Given etapas em todos os cenários em uma Feature.
- Adicionar algum contexto aos cenários que o seguem.
- Agrupar as Given etapas.

Dicas para escrita de cenários



1. Escrever os cenários em terceira pessoa

Dado que o usuário está logado
Quando o usuário exclui uma postagem no blog
Então a postagem é excluída com sucesso



Dado eu estou logado
Quando eu excluo uma postagem no blog
Então devo ver que a postagem é excluída com sucesso



Dicas para escrita de cenários



2. Escrever cenários declarativos, não imperativos

Dado que o usuário possui credenciais válidas
Quando o usuário faz o login
Então ele tem acesso aos seus dados



Dado que o usuário está na tela de login
Quando ele digita o usuário "admin"
E digita a senha "admin"
E clica em "Logar"
Então o usuário deve visualizar a sua tela inicial com os seus dados
E deve ver a mensagem "Bem-vindo Admin"



Dicas para escrita de cenários



3. Inserir uma narrativa

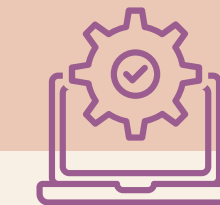
Cenário: Exibir dados pessoais do usuário

Cenário: Cadastrar um novo produto com sucesso

Cenário: Realizar login do usuário com sucesso



Dicas para escrita de cenários



4. Evite passos conjuntivos

Cenário: Cadastrar um novo produto com sucesso

Dado que o usuário tem permissão para cadastrar produtos

Quando ele preenche as informações corretamente

E envia os dados

Então o produto deve ser cadastrado com sucesso



Cenário: Cadastrar um novo produto com sucesso

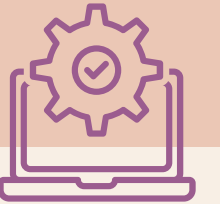
Dado que o usuário tem permissão para cadastrar produtos

Quando ele preenche as informações corretamente e envia os dados

Então o produto deve ser cadastrado com sucesso



GHERKIN - Estrutura de um Arquivo



```
# language: pt
```

Funcionalidade: Algum texto descritivo conciso do que é desejado

A fim de realizar um valor de negócio

Como ator explícito do sistema

Ele quer ganhar algum resultado benéfico que promova a meta

Texto adicional...

Cenário: Uma determinada situação de negócios

Dado uma pré condição

E uma outra pré condição

Quando uma ação é feita pelo ator

E uma outra ação

E outra ação diferente

Então um resultado testável é alcançado

E outra coisa que possamos verificar também acontece

Cenário: Uma situação diferente



arquivo no formato
.feature

GHERKIN - Realizar Login



language: pt

Funcionalidade: Login

Como um usuário do sistema ABC

Fulano quer completar o Login

Para que ele tenha acesso às funcionalidades de usuário logado

Cenário: Login válido

Dado que "Fulano" possui uma conta no sistema

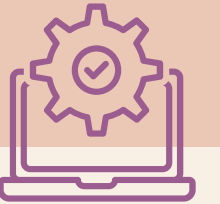
E ele acessa a página de Login

E ele preenche suas credenciais válidas

Quando ele aciona a opção para logar

Então ele deve ser redirecionado para a página inicial de usuário logado.

GHERKIN - Realizar Login com Contexto



language: pt

Funcionalidade: Login

Como um usuário do sistema ABC

Fulano quer completar o Login

Para que ele tenha acesso às funcionalidades de usuário logado

Contexto:

Dado que "Fulano" possui uma conta no sistema

Cenário: Login válido

E ele acessa a página de Login

E ele preenche suas credenciais válidas

Quando ele aciona a opção para logar

Então ele deve ser redirecionado para a página inicial de usuário logado.

Cenário: Login com senha incorreta

E ele acessa a página de Login

E ele preenche suas credenciais com senha incorreta

Quando ele aciona a opção para logar

Então uma mensagem de aviso de senha incorreta deve ser exibida

GHERKIN - Realizar Login com Esquema de Cenário



```
# language: pt
```

Funcionalidade: Login

Como um usuário do sistema ABC

Fulano quer completar o Login

Para que ele tenha acesso às funcionalidades de usuário logado

Contexto:

Dado que "Fulano" possui uma conta no sistema

Esquema de Cenário: Login com dados incorretos

E ele acessa a página de Login

E ele preenche suas credenciais com senha <situacao>

Quando ele aciona a opção para logar

Então uma mensagem de aviso de senha <situacao> deve ser exibida

Exemplos:

```
|situacao      |  
|"expirada"    |  
|"incorreta"   |
```

GHERKIN - Palavras-chave Secundárias



- Existem também algumas palavras-chave secundárias:

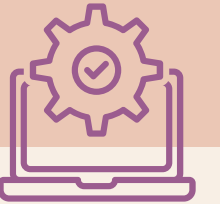
01 """"
(Doc Strings)

02 |
(Data Tables)

03 @
(Tags)

04 #
(Comentários)

Palavras-chave Secundárias

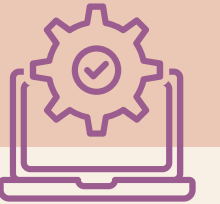


- Argumentos da Entrada
 - passar mais dados para uma etapa do que caber em uma única linha.

Doc String
" " " " " "

- passar um trecho maior de texto para uma definição de etapa.
- O texto deve ser deslocado por delimitadores que consistem em três aspas duplas em suas próprias linhas.

Palavras-chave Secundárias



|
(Data Tables)

- passar uma lista de valores para uma definição de etapa
- Escape da célula da tabela
 - \n: um caractere de nova linha em uma célula da tabela.
 - Se você precisa de um
 - \| : | como parte da célula
 - \\: \.

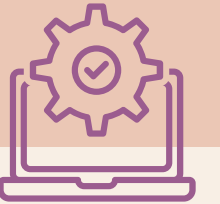
Palavras-chave Secundárias



@
(Tags)

- servem para organizar os cenários utilizando os critérios mais diversos possíveis, como por exemplo prioridade do teste, se é negativo ou positivo, etc.
- podem ser especificadas na hora da execução dos testes, para que todos os cenários que possuam a tag mencionada sejam executados.
- qualquer **tag** pode ser colocada nos cenários.
- quando colocada na funcionalidade, será aplicada para todos os cenários existentes naquela feature.

TAGs



language: pt

@login

Funcionalidade: Login

Como um usuário do sistema ABC

Fulano quer completar o Login

Para que ele tenha acesso às funcionalidades de usuário logado

Contexto:

Dado que "Fulano" possui uma conta no sistema

@positivo

Cenário: Login válido

E ele acessa a página de Login

E ele preenche suas credenciais válidas

Quando ele aciona a opção para logar

Então ele deve ser redirecionado para a página inicial de usuário logado.

@negativo

Cenário: Login com senha incorreta

E ele acessa a página de Login

E ele preenche suas credenciais com senha incorreta

Quando ele aciona a opção para logar

GHERKIN - Pagamento com Cartão de crédito



#language: pt

Funcionalidade: Pagamento com cartão de crédito

O cliente pode optar pelo pagamento com cartão de crédito ao finalizar a compra. O pagamento via cartão de crédito deve ser autorizado pela operadora. No caso do pagamento por um cartão de crédito ser negado pela operadora, o cliente deve ter a opção de selecionar outro cartão de crédito.

Contexto:

Dado que o "Beltrano" selecionou o produto "Novo Echo Dot (4ª Geração)"

E autenticou com as credenciais válidas

E ele está na "Página de pagamento"

Cenário: Crédito não autorizado pela operadora de cartão de crédito

Quando "Beltrano" informa "Crédito" como meio de pagamento

- * informa os dados do cartão

- * informa a quantidade de parcelas

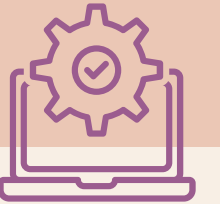
- * pressiona o botão "Finalizar compra"

E o sistema de crédito retorna "COD5000 - Crédito não autorizado"

Então "Beltrano" deve ser redirecionado para a "Página de pagamento"

E o sistema exibe a mensagem "A transação não foi autorizada pela operadora de crédito."

GHERKIN - Nível de Assinatura (imperativo)



#language: pt

Funcionalidade: os assinantes veem diferentes conjuntos de imagens de estoque com base em seu nível de assinatura

Cenário: assinantes gratuitos veem apenas os artigos gratuitos

Os usuários com assinatura gratuita podem acessar "ArtigoGratuito", mas não "ArtigoPremium"

Quando o usuário digita "rafaelamoreira@gmail.com" no campo de e-mail

E ele digita "Pass123" no campo de senha

E pressiona o botão "Enviar"

Então ele vê "ArtigoGratuito" na página inicial

E não vê "ArtigoPremium" na página inicial



Cenário: o assinante com uma assinatura paga pode acessar "ArtigoGratuito" e "ArtigoPremium"

Dado que Fulano está na página de login

Quando Fulano digita "rafaela.cruz@gmail.com" no campo de e-mail

E Fulano digita "Pass123" no campo de senha

E pressiona o botão "Enviar"

Então Fulano vê "ArtigoGratuito" e "ArtigoPremium" na página inicial

GHERKIN - Nível de Assinatura (declarativo)



#language: pt

Funcionalidade: os assinantes veem diferentes conjuntos de imagens de estoque com base em seu nível de assinatura

Cenário: assinantes gratuitos veem apenas os artigos gratuitos

Dado que "Rafaela" tem uma assinatura grátis

Quando "Rafaela" faz login com suas credenciais válidas

Então ela vê um artigo grátis na página inicial

Cenário: o assinante com uma assinatura paga pode acessar artigos gratuitos e pagos

Dado que "Willian" tem uma assinatura paga de nível básico

Quando "Willian" faz login com suas credenciais válidas

Então ele vê um artigo gratuito e um artigo pago na página inicial



GHERKIN - Calculadora



#language: pt

Funcionalidade: os usuários vem os resultados de acordo com a operação desejada

Cenário: mostrar o resultado quando executar uma operação na calculadora

Dado que "Maria" esteja na calculadora

Quando pressionar o <número1>

E pressionar o <número2>

E escolher a "<operacao>"

Então o sistema devera mostrar o <resultado>

Exemplo:

	numero1		numero2		operacao		resultado	
	4		2		soma		6	
	4		2		subtração		2	
	4		2		multiplicação		8	
	4		2		divisão		2	

Cucumber



Cucumber



O Cucumber lê especificações executáveis escritas em texto simples e valida se o software faz o que essas especificações dizem. As especificações consistem em vários exemplos ou cenários.



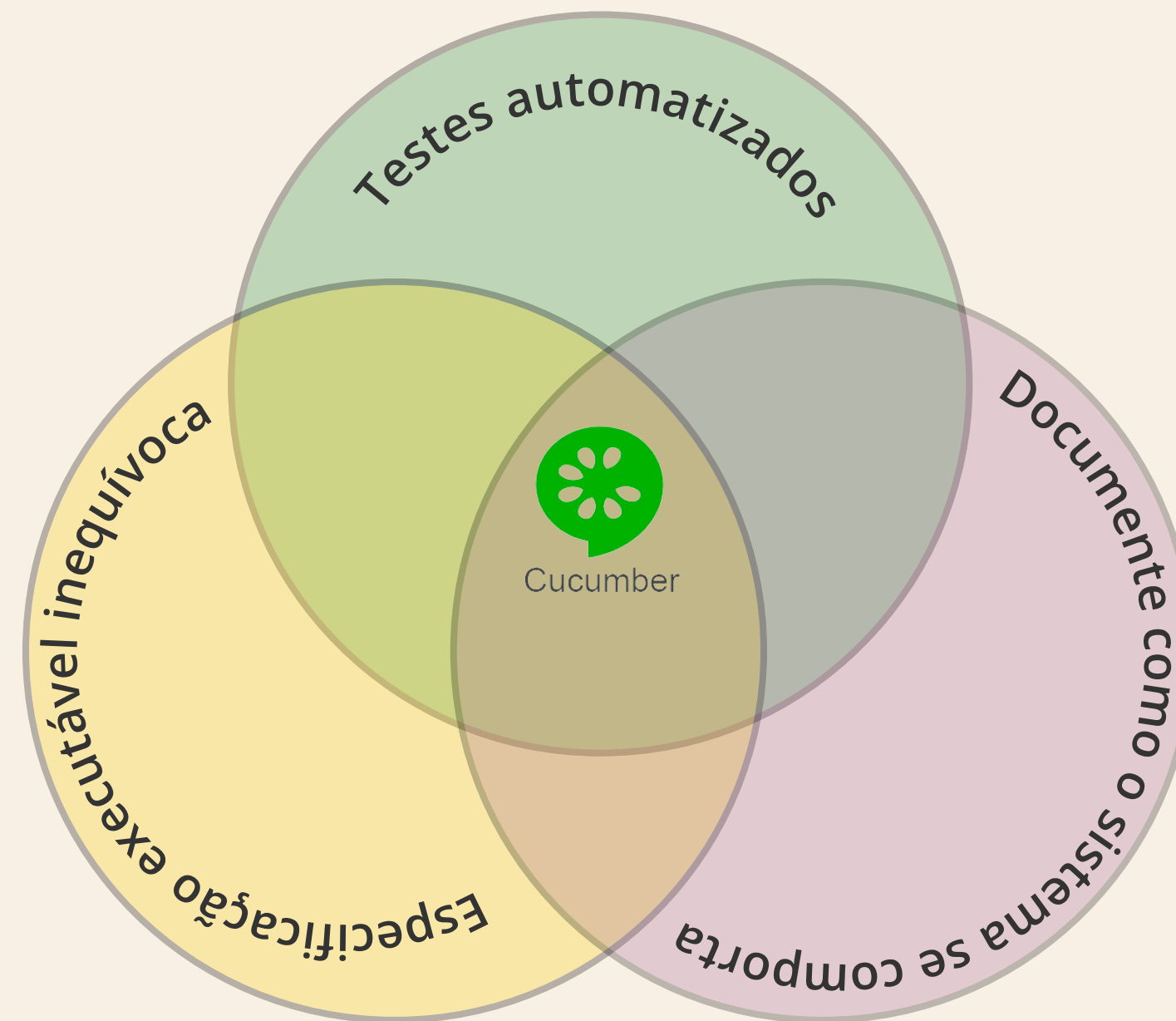
Cucumber verifica se o software está em conformidade com a especificação e gera um relatório indicando  sucesso ou  falha para cada cenário.

Cucumber



Cucumber

- O Cucumber serve a vários propósitos:



Cucumber - Sintaxe Java



Cucumber

```
@Dado("que \"Fulano\" digitou {int} e {int} na calculadora")
    public void fulano_digitou_na_calculadora(int a, int b) {
        // implementar a lógica
    }
```

```
@Quando("\"Fulano\" escolher somar")
    public void fulano_escolher_somar() {
        // implementar a lógica
    }
```

```
@Entao("o resultado devera ser {int} na tela")
    public void o_resultado_devera_aparecer_na_tela(int expectedResult) {
        // implementar a lógica
    }
```

Bora colocar a
mão na massa?



DÚVIDAS?



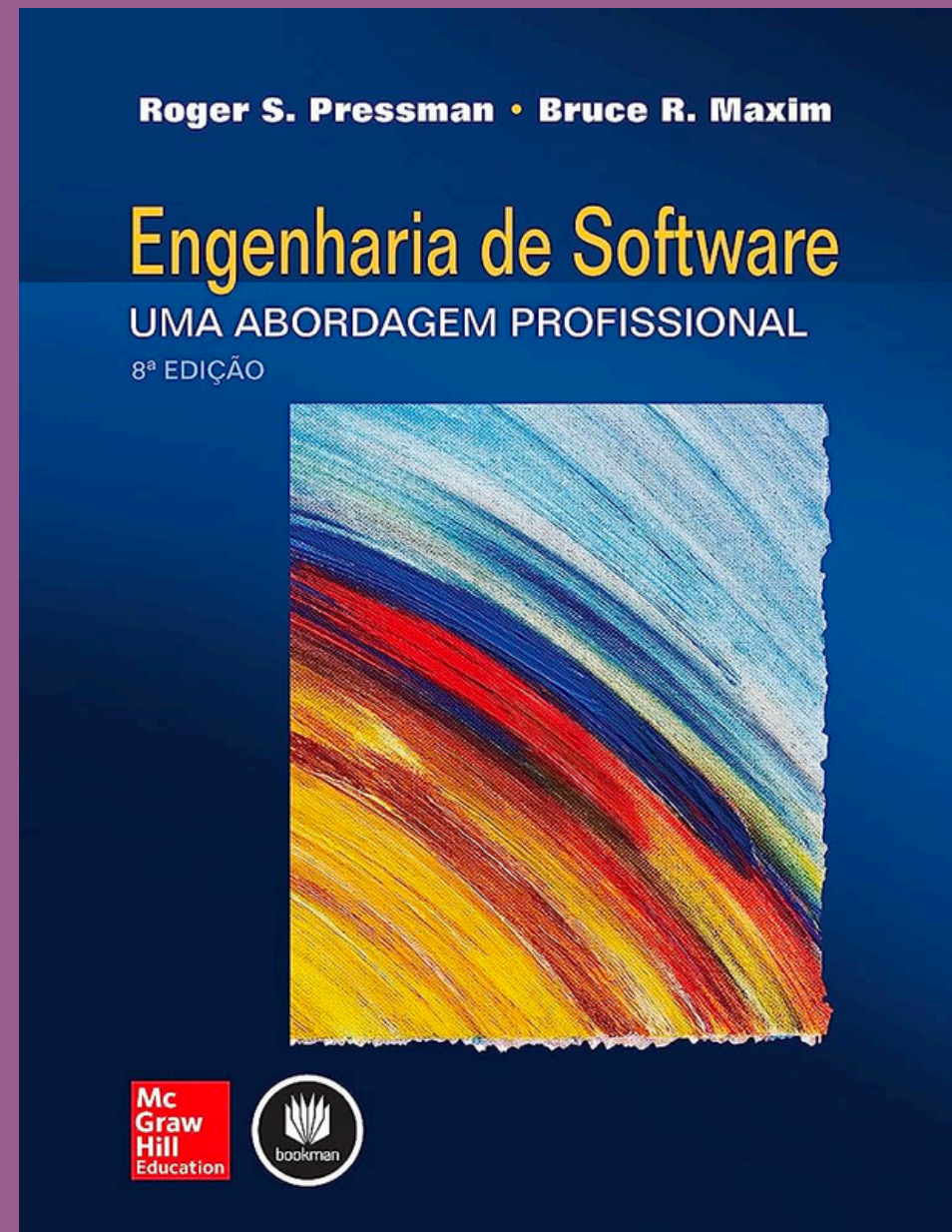
HANDS-ON



1. Criar um cenário para mostrar uma mensagem de sucesso quando um usuário for criado.
2. Crie um cenário para Pesquisa por produto.
3. Crie um cenário para Adicionar item ao carrinho.
4. Crie um cenário para Calcular subtotal no carrinho.
5. Crie um cenário para Realizar pagamento.



Referências Bibliográficas



<https://cucumber.io/docs/gherkin/reference/>

<https://medium.com/revista-tsipi/gherkin-o-dia-em-que-entendi-que-estava-escrevendo-errado-220a84520819>

PROFESSORA: RAFAELA PRISCILA CRUZ MOREIRA

CONTATOS



rafaela.cruz@ulife.com.br



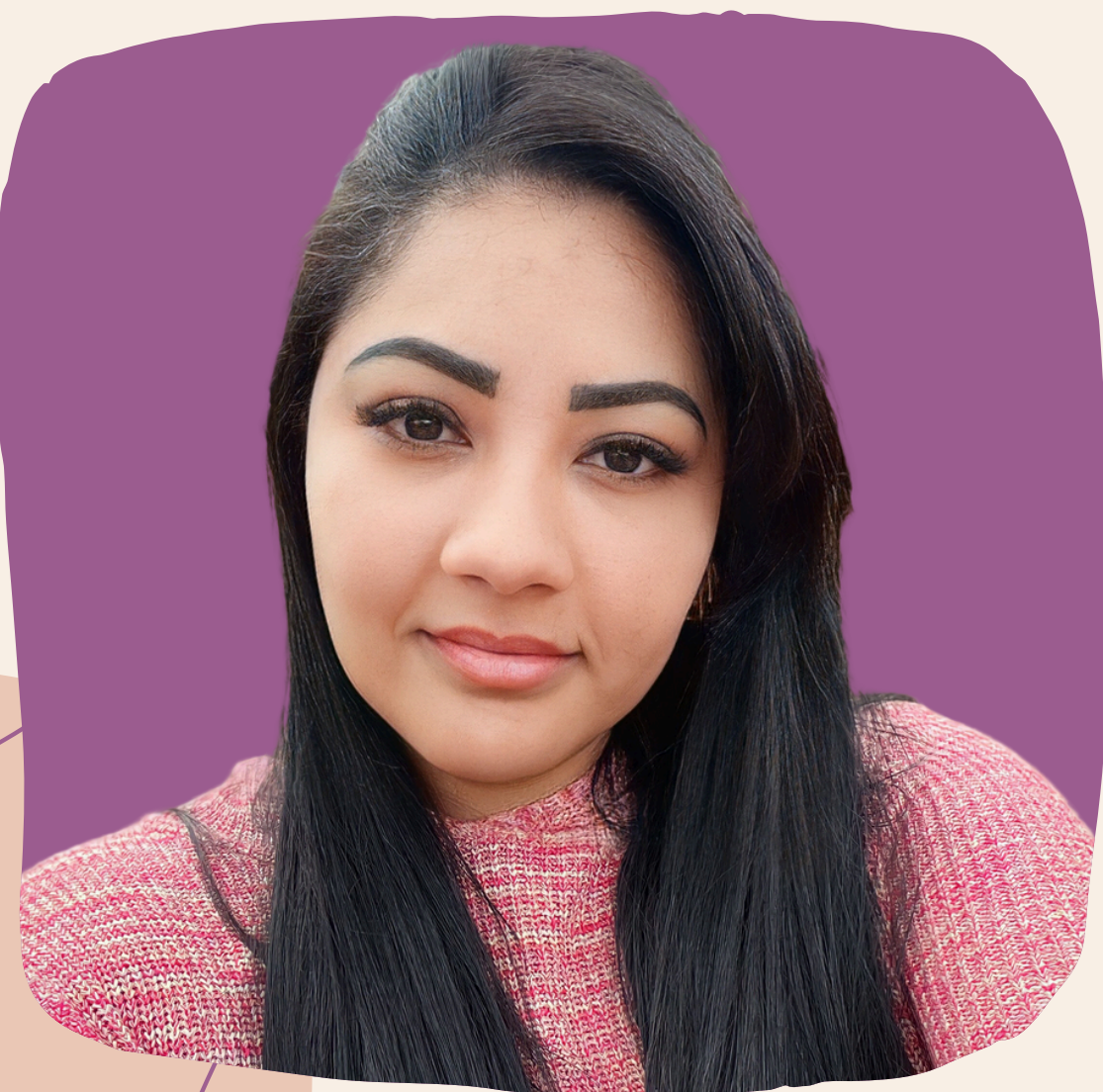
[rafaelapcmoreira](#)



[rafaelapcm](#)



[rafapcmor](#)





Thank You